



INSA SUMMER CAMP

Department of EMBEDDED SYSTEM

Sleep Mode Assignment

Name: Haimanot Desie

ID: CTC-2130-26

Submitted to: **Gutenber**

Submission date: **Sep, 14, 2026**

Table of Contents

<u>1. Objective</u>	1
<u>2. Theoretical Background & Architecture</u>	1
2.1 Dynamic Power Dissipation.....	1
2.2 ATmega328P Sleep Modes.....	1
2.3 Peripheral & Timer Behavior in Power-Down.....	1
2.4 Interrupt Wake-Up vs. Hardware Reset.....	1
<u>3. Register Configuration & Bit Explanations</u>	2
<u>4. Hardware Wiring & Proteus Pin Connections</u>	2
<u>5. System Flowchart</u>	3
<u>6. Simulation Demonstration & Operating States</u>	4
6.1 Condition 1 — Normal Operation.....	4
6.2 Condition 2 — Sleep Mode Entry.....	4
6.3 Condition 3 — Interrupt-Based Wake-Up.....	5
<u>7. Firmware Source Code</u>	7
<u>8. Conclusion</u>	11

1. Objective

The objective of this assignment is to design, model, and simulate an autonomous, low power embedded control system based on the ATmega328P microcontroller. The system minimizes dynamic CMOS power dissipation by transitioning into **Power-down Sleep Mode** after active execution and recovers cleanly via an asynchronous external low level hardware interrupt on pin PD2 (INT0), without executing a system reset. System health and operational states are displayed on an HD44780-compatible 16x2 LCD, a status LED, and an audible piezoelectric buzzer.

2. Theoretical Background & Architecture

2.1 Dynamic Power Dissipation

Dynamic power dissipation in CMOS circuits is governed by the classical relationship:

$$P_{dynamic} = C_{load} \times (V_{DD})^2 \times f_{clock}$$

By gating off high frequency clock distribution trees ($f_{clock} \rightarrow 0$ Hz), dynamic switching losses drop to zero, leaving only static silicon leakage current (approximately 1 μ A).

2.2 ATmega328P Sleep Modes

- **Idle Mode:** Stops clk_{CPU} and clk_{FLASH} ; keeps $clk_{I/O}$, clk_{ADC} , and internal peripherals (Timers, USART, SPI) operational.
- **ADC Noise Reduction Mode:** Stops clk_{CPU} , clk_{FLASH} , and $clk_{I/O}$; keeps clk_{ADC} operational to eliminate digital noise during analog sampling.
- **Power-down Mode:** The deepest power-saving mode. All internal clock trees and the main quartz oscillator are halted. Only asynchronous modules (INT0/INT1 low-level, PCINT, WDT) remain active.
- **Power-save Mode:** Identical to Power-down, but keeps Timer2 running asynchronously if driven by an external 32.768 kHz watch crystal.
- **Standby Mode:** Identical to Power-down, but the external crystal oscillator remains running, allowing wake-up recovery in 6 clock cycles.

2.3 Peripheral & Timer Behavior in Power-Down

- Synchronous timers (Timer0, Timer1) freeze because $clk_{I/O}$ is halted. They retain register values statically but cannot count or trigger interrupts.
- Pin PD2 (INT0) operates asynchronously via static combinatorial gates, triggering the wake-up controller without an active clock.

2.4 Interrupt Wake-Up vs. Hardware Reset

- An interrupt wake-up restarts the oscillator, executes $ISR(INT0_{vect})$, and returns via RETI to the instruction immediately following $sleep_cpu()$. SRAM variables, stack memory, and peripheral registers remain intact.

- A reset forces the Program Counter to 0x0000 and reinitializes memory sections from scratch.

3. Register Configuration & Bit Explanations

The following registers were configured according to the ATmega328P datasheet to implement Power-down sleep with an asynchronous, low-level INT0 wake-up source.

Table 1. Register configuration and bit-level explanation.

Register	Bit(s)	Value / Action	Purpose
SMCR	SM2:SM0	0,1,0	Selects Power-down Sleep Mode.
SMCR	SE (Bit 0)	1 → sleep_cpu() → 0	Software interlock: set immediately before sleep_cpu(), cleared immediately after wake-up to prevent accidental re-entry.
EICRA	ISC01, ISC00	0, 0	Configures low-level sensing for INT0, which functions asynchronously while clk _{IO} is halted.
EIMSK	INT0 (Bit 0)	1	Unmasks the external interrupt channel on physical pin PD2.
EIFR	INTF0 (Bit 0)	1 (write to clear)	Clears any lingering sticky interrupt flag before sleeping.
SREG	I (Bit 7)	1 via sei()	Enables global CPU interrupt servicing.
DDRD / PORTD	DDD2 = 0, PORTD2 = 1	Input, pull-up enabled	Configures PD2 as an input with the internal pull-up resistor active (idle state = 5 V).

4. Hardware Wiring & Proteus Pin Connections

Table 2. Hardware wiring and Proteus pin connections.

ATmega328P Pin	Function	Connects To
Pin 32	PD2 / INT0	Momentary push-button (SW1) to GND, pull-up active
Pin 12	PB0	220 Ω resistor → Red LED (D1) → GND
Pin 13	PB1	5 V active piezo buzzer (BUZ1) → GND
Pin 14	PB2	LCD RS (Pin 4)
Pin 15	PB3	LCD E (Pin 6)
Pin 23	PC0	LCD D4 (Pin 11)
Pin 24	PC1	LCD D5 (Pin 12)
Pin 25	PC2	LCD D6 (Pin 13)
Pin 26	PC3	LCD D7 (Pin 14)
Pins 7 & 8	XTAL1, XTAL2	16.0 MHz crystal with two 22 pF capacitors to GND
Pin 29	PC6 / RESET	10 k Ω resistor to +5 V

5. System Flowchart

The flowchart below summarizes the control logic implemented in firmware: normal operation, entry into Power-down sleep, and asynchronous wake-up via the INT0 interrupt, with the program looping back to normal operation without a system reset.

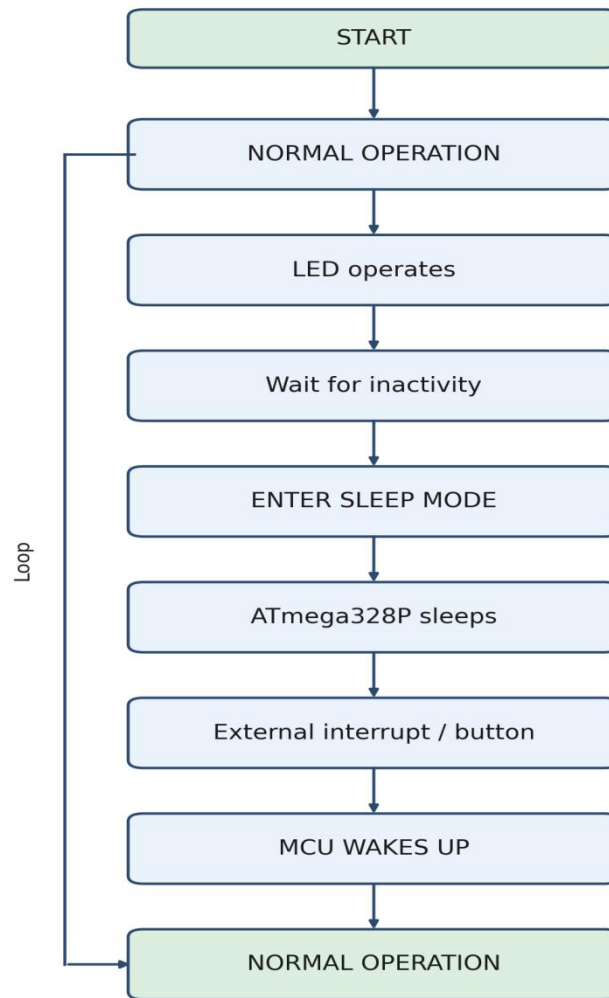


Figure 1. Firmware control flow: active operation → sleep entry → INT0 wake-up → resume.

6. Simulation Demonstration & Operating States

6.1 Condition 1 — Normal Operation

- LCD displays: "SYSTEM ACTIVE" on Row 1, "NORMAL MODE" on Row 2.
- LED D1 on PB0 is lit (logic HIGH).
- CPU core and $\text{clk}_{I/O}$ are running at 16 MHz.

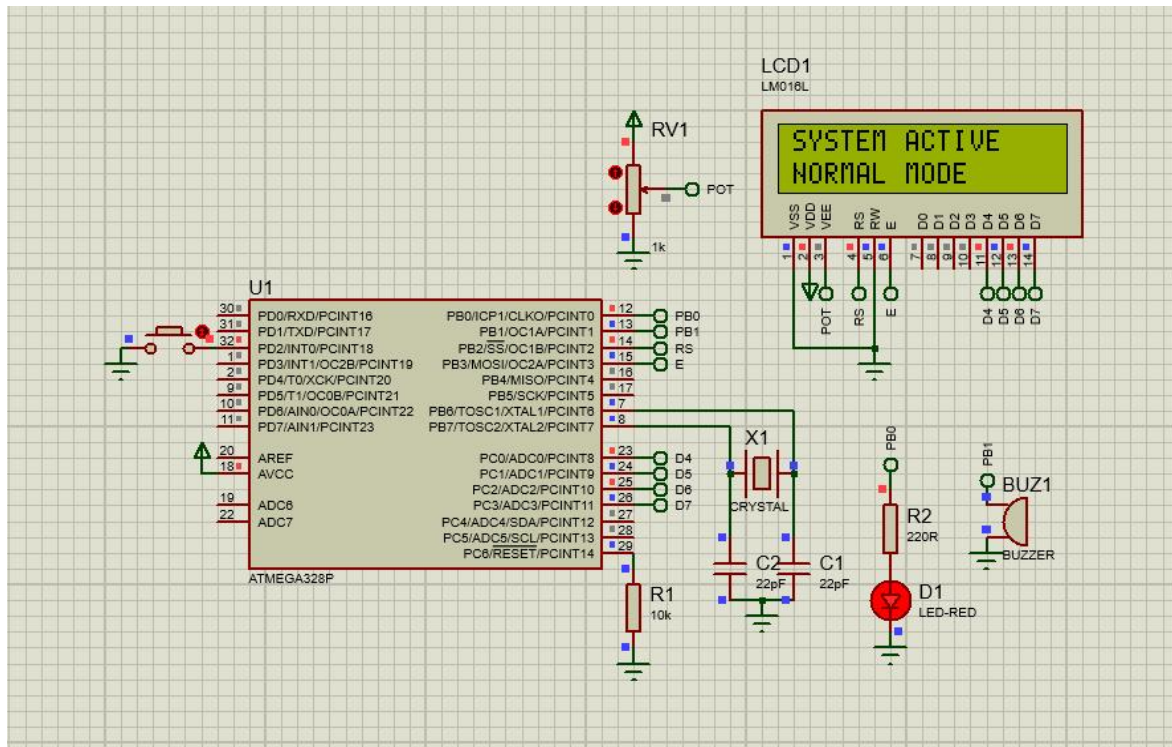


Figure 2. Proteus schematic — normal operation (LCD: SYSTEM ACTIVE / NORMAL MODE, LED D1 ON).

6.2 Condition 2 — Sleep Mode Entry

- LCD displays: "ENTERING SLEEP" on Row 1, "POWER-DOWN" on Row 2.
- LED D1 is turned OFF (PB0 driven LOW).
- SE bit is set, `sleep_cpu()` executes, the oscillator stops, and all synchronous clocks halt.
- Current draw drops to approximately 1 μ A.

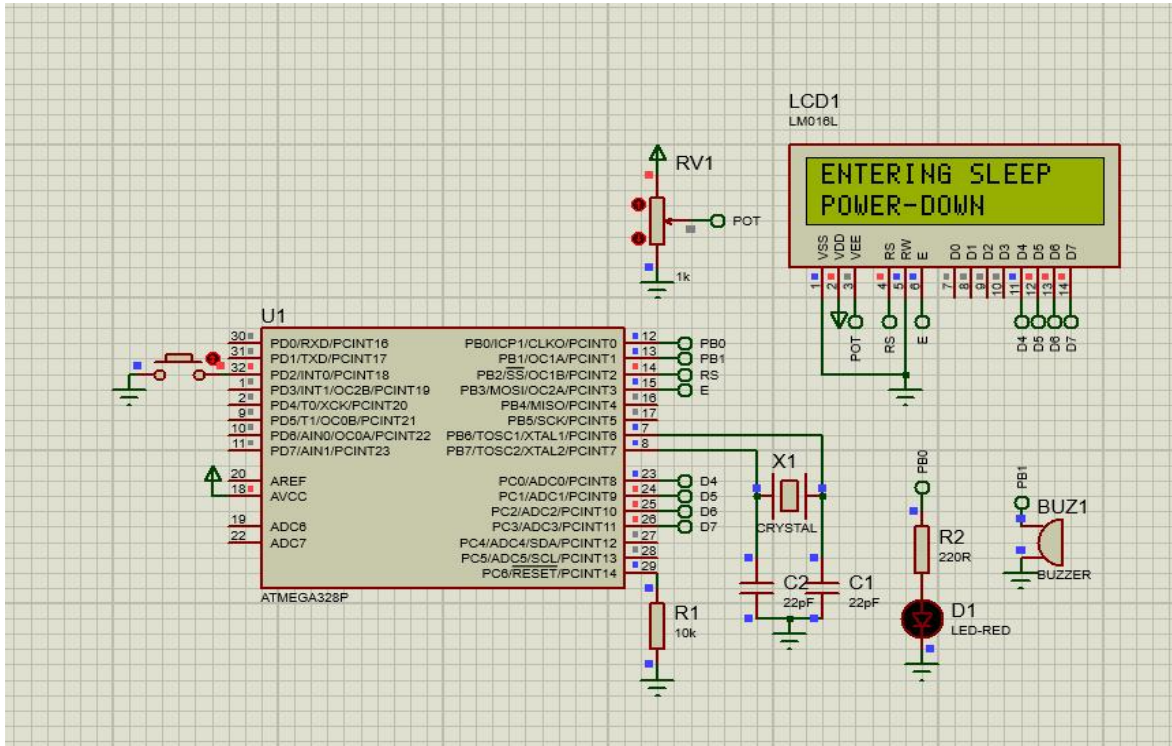


Figure 3. Proteus schematic — Power-down sleep entry (LCD: ENTERING SLEEP / POWER-DOWN, LED D1 OFF).

6.3 Condition 3 — Interrupt-Based Wake-Up

- Button SW1 is pressed, pulling PD2 to GND.
- The asynchronous comparator restarts the crystal oscillator.
- The core jumps to ISR(INT0_vect), sets wake_up = 1, and returns via RETI.
- Execution resumes at the instruction immediately following sleep_cpu(), clearing the SE bit.
- Buzzer BUZ1 chirps, LED D1 turns back on, and the LCD displays "MCU AWAKE" / "INT0 WAKE-UP".
- No system reset occurs; execution loops cleanly back to Condition 1.

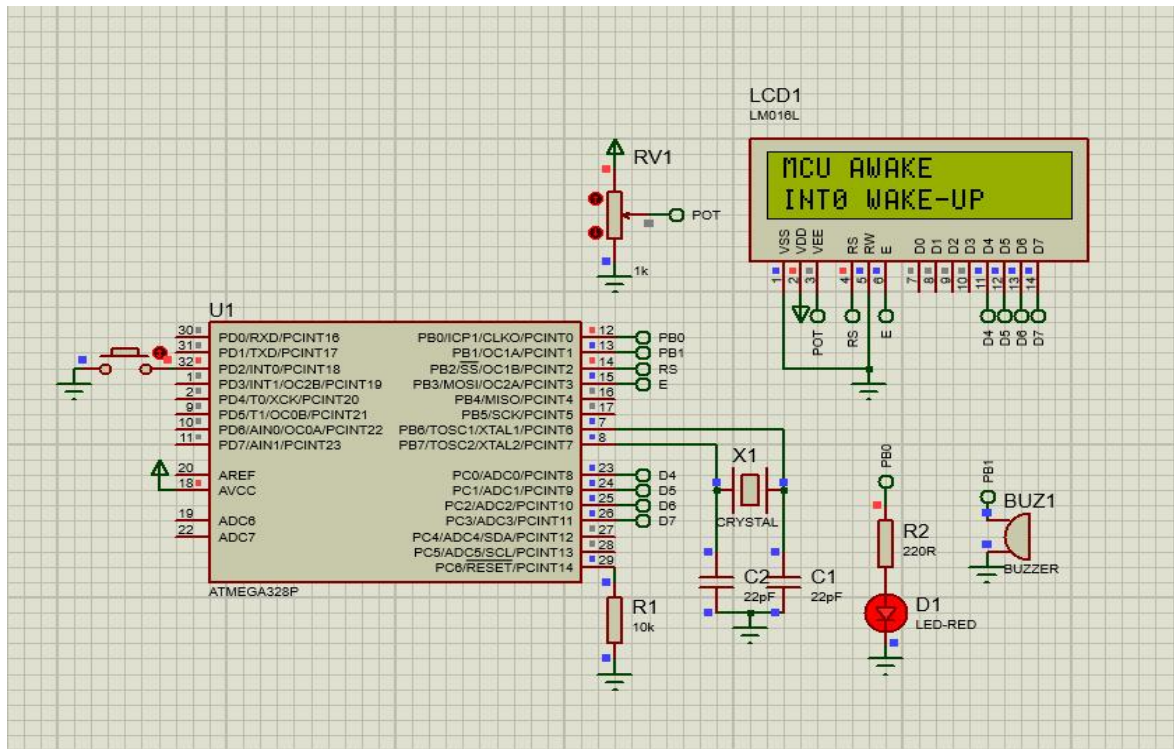


Figure 4. Proteus schematic — INT0 wake-up (LCD: MCU AWAKE / INT0 WAKE-UP).

7. Firmware Source Code

The complete AVR-GCC source code implementing LCD control, INT0 configuration, Power-down sleep entry, and interrupt-driven wake-up is listed below.

```
#define F_CPU 16000000UL

#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <util/delay.h>
volatile unsigned char wake_up = 0;
void LCD_Enable(void)
{
    PORTB |= (1 << PB3);
    PORTB &= ~(1 << PB3);
}
void LCD_Send4Bit(unsigned char data)
{
    PORTC &= 0xF0;
    PORTC |= (data & 0x0F);
    LCD_Enable();
}
void LCD_Command(unsigned char command)
{
    PORTB &= ~(1 << PB2);
    LCD_Send4Bit(command >> 4);
    LCD_Send4Bit(command & 0x0F);
    if (command == 0x01 || command == 0x02)
        _delay_ms(2);
    else
        _delay_us(50);
}
void LCD_Data(unsigned char data)
{
    PORTB |= (1 << PB2);
    LCD_Send4Bit(data >> 4);
    LCD_Send4Bit(data & 0x0F);
    _delay_us(50);
}
void LCD_Init(void)
{
    DDRB |= (1 << PB2);
    DDRB |= (1 << PB3);
    DDRC |= (1 << PC0);
    DDRC |= (1 << PC1);
    DDRC |= (1 << PC2);
    DDRC |= (1 << PC3);
    _delay_ms(20);
    PORTB &= ~(1 << PB2);
    LCD_Send4Bit(0x03);
    _delay_ms(5);
}
```

```

    LCD_Send4Bit(0x03);
    _delay_us(150);
    LCD_Send4Bit(0x03);
    _delay_us(150);
    LCD_Send4Bit(0x02);
    LCD_Command(0x28);
    LCD_Command(0x0C);
    LCD_Command(0x06);
    LCD_Command(0x01);
    _delay_ms(2);
}

void LCD_Clear(void)
{
    LCD_Command(0x01);
    _delay_ms(2);
}

void LCD_String(const char *text)
{
    while (*text)
    {
        LCD_Data(*text);
        text++;
    }
}

void LCD_SetCursor(unsigned char row, unsigned char column)
{
    if (row == 0)
        LCD_Command(0x80 + column);
    else
        LCD_Command(0xC0 + column);
}

ISR(INT0_vect)
{
    wake_up = 1;
}

void INT0_Init(void)
{
    DDRD &= ~(1 << PD2);
    PORTD |= (1 << PD2);
    EICRA &= ~(1 << ISC01);
    EICRA &= ~(1 << ISC00);
    EIFR |= (1 << INTF0);
    EIMSK |= (1 << INT0);
}

void Sleep_Init(void)
{
    SMCR &= ~(1 << SM2);
    SMCR |= (1 << SM1);
    SMCR &= ~(1 << SM0);
}

void Enter_Sleep(void)
{
    LCD_Clear();

```

```

    LCD_SetCursor(0, 0);
    LCD_String("ENTERING SLEEP");
    LCD_SetCursor(1, 0);
    LCD_String("POWER-DOWN");
    PORTB &= ~(1 << PB0);
    PORTB &= ~(1 << PB1);
    wake_up = 0;
    EIFR |= (1 << INTF0);
    SMCR |= (1 << SE);
    sleep_cpu();
    SMCR &= ~(1 << SE);
}

int main(void)
{
    DDRB |= (1 << PB0);
    DDRB |= (1 << PB1);
    PORTB &= ~(1 << PB0);
    PORTB &= ~(1 << PB1);
    LCD_Init();
    INT0_Init();
    Sleep_Init();
    sei();
    while (1)
    {
        LCD_Clear();
        LCD_SetCursor(0, 0);
        LCD_String("SYSTEM ACTIVE");
        LCD_SetCursor(1, 0);
        LCD_String("NORMAL MODE");
        PORTB |= (1 << PB0);
        _delay_ms(500);
        Enter_Sleep();
        if (wake_up == 1)
        {
            PORTB |= (1 << PB0);
            PORTB |= (1 << PB1);
            PORTB &= ~(1 << PB1);
            LCD_Clear();
            LCD_SetCursor(0, 0);
            LCD_String("MCU AWAKE");
            LCD_SetCursor(1, 0);
            LCD_String("INT0 WAKE-UP");
            _delay_ms(100);
            wake_up = 0;
            while (!(PIND & (1 << PD2)))
            {
            }
            _delay_ms(50);
        }
    }
}

```

8. Conclusion

The Proteus simulation validates bare-metal low-power management on the ATmega328P. Configuring Power-down Mode via SMCR eliminates dynamic power dissipation by halting internal clocks. Using asynchronous low-level interrupt sensing on INT0 guarantees reliable wake-up recovery without resetting the microcontroller, preserving runtime context and SRAM integrity. The three demonstrated conditions — normal operation, sleep entry, and interrupt-driven wake-up — confirm that the firmware and hardware design meet the stated objective: the MCU resumes execution immediately after `sleep_cpu()` rather than restarting from the reset vector, and all peripheral states (LED and LCD) reflect the correct operating mode at each stage.